

PIPELINED RISC-V
IMPLEMENTATION
USING VERILOG

PREPARED BY:

Nensi Thummar

Contents

1. ABSTRACT	4
2. INTRODUCTION	4
2.1 INTRODUCTION TO RISC-V	4
2.2 INTRODUCTION TO PIPELINING	5
2.3 NEED FOR PIPELINING	5
3. OBJECTIVES	5
4. SYSTEM ARCHITECTURE	6
4.1 OVERALL BLOCK DIAGRAM	6
4.2 FIVE PIPELINE STAGES	7
5. DESIGN MODULES	8
5.1 Arithmetic Logic Unit (ALU)	8
5.2 Register File	8
5.3 Instruction Memory	9
5.4 Data Memory	9
5.5 Controller	9
5.6 Sign Extension Unit	10
5.7 Pipeline Registers	10
5.8 Program Counter (PC)	11
5.9 PC+4 Adder	11
5.10 PC Target Adder	11
5.11 ALU Source Multiplexer (ALUSrc MUX)	11
5.12 Result Multiplexer	11
6. HAZARDS	12
6.1 Data Hazards	12
6.2 Control Hazards	12
7. HAZARD HANDLING MECHANISMS	13
7.1 Forwarding Unit	13
7.2 Hazard Detection Unit	13
8. CONTROL SIGNALS	14
9. COMMON RISC-V INSTRUCTIONS	15
9.1 R-Type Instructions	15
9.2 I-Type Instructions	16
9.3 S-Type Instructions	16
9.4 B-Type Instructions	17
9.5 U-Type Instructions	17

9.6 J-Type Instructions.....	17
10. INSTRUCTIONS SUPPORTED BY OUR DESIGN	17
11. SIMULATION RESULTS	18
12. ADVANTAGES	20
13. LIMITATIONS	20
14. APPLICATIONS.....	20
15. CHALLENGES	20
16. LEARNINGS	21
17. CONCLUSION	21
18. REFERENCES.....	21

1. ABSTRACT

Modern processors are expected to execute programs quickly and efficiently while maintaining reliable operation. One of the most effective techniques used to improve processor performance is pipelining, where multiple instructions are processed simultaneously by dividing execution into several stages. This project presents the design and implementation of a 32-bit five-stage pipelined RISC-V processor using Verilog Hardware Description Language (HDL).

The designed processor is based on the RV32I instruction set architecture and consists of the Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. Various functional modules such as the Arithmetic Logic Unit (ALU), Register File, Control Unit, Instruction Memory, Data Memory, and Sign Extension Unit have been integrated to achieve complete processor functionality. To ensure correct execution of instructions in a pipelined environment, hazard management techniques including a Forwarding Unit and Hazard Detection Unit have been incorporated to handle data and control hazards effectively.

The design was simulated and verified using a dedicated Verilog testbench. The simulation results confirmed correct execution of arithmetic, logical, memory, branch, and jump instructions. The project demonstrates the effectiveness of pipelining and hazard handling techniques in improving processor performance.

2. INTRODUCTION

A processor executes instructions stored in memory to perform arithmetic, logical, control, and data transfer operations. The performance of a processor depends on how efficiently these instructions are executed. RISC-V is a modern open-source Instruction Set Architecture (ISA) that provides a simple and flexible platform for processor design. To improve execution speed and increase instruction throughput, pipelining is commonly used in processor architectures.

This project implements a 32-bit five-stage pipelined RISC-V processor using Verilog HDL. The processor supports the RV32I instruction set and consists of Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write Back stages. Hazard detection and forwarding mechanisms are incorporated to ensure correct execution of instructions in a pipelined environment.

2.1 INTRODUCTION TO RISC-V

RISC-V is an open-source Reduced Instruction Set Computer (RISC) Instruction Set Architecture developed at the University of California, Berkeley. It provides a simple and modular instruction set that can be freely used, modified, and extended without licensing restrictions.

RISC-V follows the RISC design philosophy, which uses simple instructions that can be executed efficiently by hardware. Due to its flexibility and openness, RISC-V has become widely used in education, research, embedded systems, and commercial processor development.

2.2 INTRODUCTION TO PIPELINING

Pipelining is a technique used to improve processor performance by dividing instruction execution into multiple stages. Instead of waiting for one instruction to complete before starting the next, multiple instructions are processed simultaneously in different stages of the pipeline.

In a five-stage pipeline, the stages are:

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)
- Memory Access (MEM)
- Write Back (WB)

RISC-V Pipeline Stages



Fig.1: Five-Stage Pipeline Architecture

This allows the processor to increase instruction throughput and utilize hardware resources more efficiently.

2.3 NEED FOR PIPELINING

In a non-pipelined processor, only one instruction is executed at a time, resulting in lower performance and inefficient hardware utilization.

Pipelining is required to:

- Increase instruction throughput
- Improve processor performance
- Reduce idle time of hardware units
- Allow multiple instructions to execute simultaneously
- Achieve faster program execution

As modern applications require higher processing speeds, pipelining has become a fundamental feature of processor architectures.

3. OBJECTIVES

The objectives of this project are:

- To design a 32-bit RV32I RISC-V processor using Verilog HDL.
- To implement a five-stage pipelined architecture.
- To design modules such as ALU, Register File, Control Unit, Instruction Memory, and Data Memory.
- To implement hazard detection and forwarding mechanisms.
- To verify processor functionality using simulation and testbench analysis.
- To study the working of pipelined processor architectures and RISC-V instruction execution.

4. SYSTEM ARCHITECTURE

The proposed processor is a 32-bit five-stage pipelined RISC-V processor based on the RV32I instruction set architecture. The processor executes instructions by dividing the execution process into five stages, allowing multiple instructions to be processed simultaneously. The processor consists of the following major modules:

- Arithmetic Logic Unit (ALU)
- Register File
- Instruction Memory
- Data Memory
- Controller
- Sign Extension Unit
- Pipeline Registers
- Program Counter (PC)
- PC+4 Adder
- PC Target Adder
- ALU Source Multiplexer (ALUSrc MUX)
- Result Multiplexer
- Forwarding Unit
- Hazard Detection Unit

The five pipeline stages are connected through pipeline registers namely IF/ID, ID/EX, EX/MEM, and MEM/WB. These registers store intermediate data and control signals between consecutive stages. The pipeline registers isolate consecutive stages and allow multiple instructions to execute simultaneously without interference.

4.1 OVERALL BLOCK DIAGRAM

The processor architecture consists of five pipeline stages connected through pipeline registers. Instructions are fetched from the Instruction Memory, decoded by the Control Unit, executed by the ALU, accessed through Data Memory when required, and finally written back to the Register File.

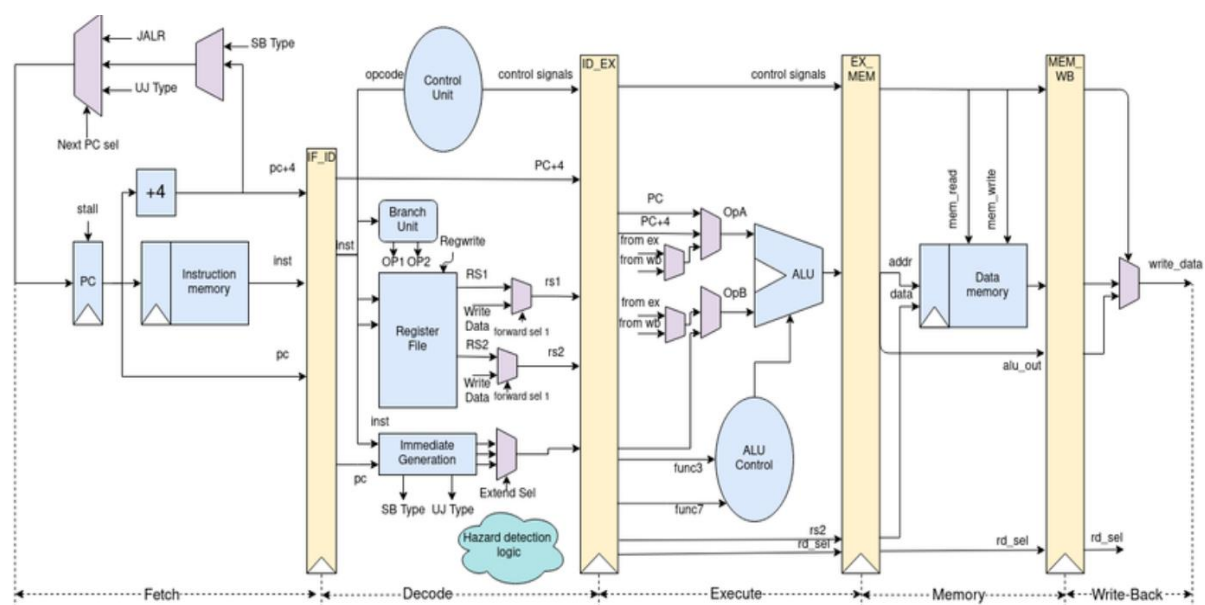


Fig.2: Pipelined RISC-V Block Diagram

4.2 FIVE PIPELINE STAGES

4.2.1 Instruction Fetch (IF) Stage

The Instruction Fetch stage retrieves instructions from the Instruction Memory using the address stored in the Program Counter (PC). The PC is normally incremented by 4 to fetch the next sequential instruction. For branch and jump instructions, the PC is updated with the target address.

Functions:

- Fetch instruction from Instruction Memory
- Generate $PC + 4$
- Update Program Counter

4.2.2 Instruction Decode (ID) Stage

The Instruction Decode stage interprets the fetched instruction and generates the required control signals. Source register operands are read from the Register File, and immediate values are generated using the Sign Extension Unit.

Functions:

- Decode instruction fields
- Read source registers
- Generate control signals
- Generate immediate values

4.2.3 Execute (EX) Stage

The Execute stage performs arithmetic, logical, comparison, and address calculation operations using the ALU. Branch decisions are also determined in this stage.

Functions:

- Perform ALU operations
- Calculate memory addresses
- Evaluate branch conditions
- Generate branch target addresses

4.2.4 Memory Access (MEM) Stage

The Memory Access stage performs load and store operations through the Data Memory. Instructions that do not require memory access simply pass through this stage.

Functions:

- Read data from memory
- Write data to memory
- Forward execution results to the next stage

4.2.5 Write Back (WB) Stage

The Write Back stage updates the destination register with the final result. The data written back may come from the ALU, Data Memory, immediate value, or $PC + 4$ depending on the instruction type.

Functions:

- Write ALU result to Register File
- Write memory data to Register File
- Complete instruction execution

5. DESIGN MODULES

The proposed pipelined RISC-V processor is constructed using several hardware modules that work together to execute instructions efficiently. Each module performs a specific function within the processor datapath and control path.

5.1 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is responsible for performing arithmetic and logical operations required during instruction execution. It receives operands from the Register File or immediate values and generates the corresponding result.

The ALU supports operations such as:

- Addition (ADD)
- Subtraction (SUB)
- Bitwise AND
- Bitwise OR
- Bitwise XOR
- Set Less Than (SLT)
- Shift Left Logical (SLL)
- Shift Right Logical (SRL)
- Shift Right Arithmetic (SRA)

The ALU also generates comparison results used for branch instructions such as BEQ, BNE, BLT, and BGE.

5.2 Register File

The Register File stores temporary data required during instruction execution. It consists of 32 general-purpose registers, each having a width of 32 bits.

The Register File provides:

- Two read ports and one write port
- Simultaneous read and write operations

Name	Register Number	Usage
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporaries
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments / return values
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporaries

Fig.3: 32 Registers

Register x0 is permanently fixed to zero and cannot be modified. Source operands are read during the Decode stage, while results are written back during the Write Back stage.

5.3 Instruction Memory

The Instruction Memory stores the program instructions that are executed by the processor.

During the Instruction Fetch stage, the Program Counter (PC) provides the address of the instruction to be fetched. The corresponding instruction is then supplied to the pipeline for execution.

The instruction memory is initialized using a memory file containing machine code instructions.

Functions:

- Store program instructions
- Provide instruction based on PC value
- Support sequential instruction fetching

5.4 Data Memory

The Data Memory is used for load and store operations.

During execution:

- Load instructions read data from memory.
- Store instructions write data into memory.

The memory is organized as a 32-bit word-addressable memory and is accessed during the Memory stage of the pipeline.

Functions:

- Read data from memory
- Write data to memory
- Support load and store instructions

5.5 Controller

The Controller generates the control signals required for proper instruction execution.

It decodes the opcode and function fields of an instruction and determines the operation to be performed by different modules within the processor.

The generated control signals include:

- RegWrite
- MemWrite
- MemRead
- ALUSrc
- ResultSrc
- ALUControl
- ImmSrc
- Branch Control Signals
- WD3Src
- BranchOp

The Controller acts as the decision-making unit of the processor and ensures correct operation of all pipeline stages.

5.6 Sign Extension Unit

Different RISC-V instructions contain immediate values of varying sizes. The Sign Extension Unit converts these immediate fields into 32-bit values before they are used by the processor.

The unit supports:

- I-Type immediates
- S-Type immediates
- B-Type immediates
- U-Type immediates
- J-Type immediates

5.7 Pipeline Registers

Pipeline registers are placed between adjacent pipeline stages to store intermediate data and control signals.

The processor contains four pipeline registers:

IF/ID Register

Stores:

- Fetched instruction
- Program Counter value

ID/EX Register

Stores:

- Register operands
- Immediate value
- Control signals

EX/MEM Register

Stores:

- ALU result
- Memory control signals
- Destination register information

MEM/WB Register

Stores:

- Memory data
- ALU result
- Write-back control signals

5.8 Program Counter (PC)

The Program Counter (PC) stores the address of the current instruction being executed. During each clock cycle, the PC is updated with the address of the next instruction. Under normal operation, the PC increments by 4 to fetch the next sequential instruction. For branch and jump instructions, the PC is updated with the calculated target address.

5.9 PC+4 Adder

The PC+4 Adder generates the address of the next sequential instruction by adding 4 to the current Program Counter value. Since each RISC-V instruction occupies 4 bytes, incrementing the PC by 4 allows the processor to fetch consecutive instructions.

5.10 PC Target Adder

The PC Target Adder calculates branch and jump target addresses by adding the sign-extended immediate value to the current Program Counter. The generated target address is used when branch or jump instructions are taken.

5.11 ALU Source Multiplexer (ALUSrc MUX)

The ALU Source Multiplexer selects the second input operand for the ALU. Depending on the instruction type, the operand can be either register data from the Register File or an immediate value generated by the Sign Extension Unit.

5.12 Result Multiplexer

The Result Multiplexer selects the value to be written back into the Register File during the Write Back stage. The selected value may be the ALU result, memory data, PC+4, or an immediate value depending on the instruction type.

6. HAZARDS

In a pipelined processor, multiple instructions are executed simultaneously in different stages of the pipeline. Although pipelining improves performance, it can lead to situations where instructions interfere with each other. These situations are known as hazards.

To ensure correct instruction execution, the proposed RISC-V processor implements hazard handling mechanisms such as forwarding and hazard detection.

6.1 Data Hazards

A data hazard occurs when an instruction depends on the result of a previous instruction that has not yet completed execution.

For example:

ADD x1, x2, x3

SUB x4, x1, x5

In this case, the SUB instruction requires the value of register x1, which is produced by the ADD instruction. Since both instructions are present in the pipeline simultaneously, the required data may not yet be available, leading to a data hazard.

Data hazards can cause incorrect execution if not handled properly.

6.2 Control Hazards

A control hazard occurs when the processor encounters branch or jump instructions.

For example:

BEQ x1, x2, LABEL

The processor fetches subsequent instructions before determining whether the branch condition is true or false. If the branch is taken, the fetched instructions may be incorrect and must be discarded.

Control hazards mainly occur due to:

- Branch instructions (BEQ, BNE, BLT, BGE)
- Jump instructions (JAL, JALR)

These hazards can affect processor performance if not handled properly.

7. HAZARD HANDLING MECHANISMS

7.1 Forwarding Unit

The Forwarding Unit is used to reduce data hazards by forwarding results directly from later pipeline stages to the Execute stage without waiting for register write-back. Forwarding resolves most RAW (Read After Write) data hazards without introducing pipeline stalls.

The Forwarding Unit is implemented to resolve data hazards that occur when an instruction depends on the result of a previous instruction still present in the pipeline. Instead of waiting for the result to be written back into the Register File, the required data is directly forwarded from a later pipeline stage to the Execute stage.

The unit continuously compares the source registers (rs1 and rs2) of the instruction in the Execute stage with the destination registers (rd) of instructions in the EX/MEM and MEM/WB stages. If a match is detected and the destination register is valid, forwarding control signals are generated. These signals select the forwarded value through multiplexers connected to the ALU inputs.

Instead of stalling the pipeline, the required operand is obtained from:

- EX/MEM pipeline register
- MEM/WB pipeline register

Functions of the Forwarding Unit:

- Detect data dependencies
- Forward recent results to the ALU inputs
- Reduce pipeline stalls
- Improve processor performance

7.2 Hazard Detection Unit

The Hazard Detection Unit handles hazards that cannot be resolved using forwarding alone. The Hazard Detection Unit is responsible for identifying hazards that may affect correct instruction execution. In the design, the unit handles load-use hazards and control hazards caused by branch and jump instructions.

For load-use hazards, the unit compares the source registers of the instruction in the Decode stage with the destination register of a load instruction in the Execute stage. If a dependency is detected, the Program Counter and IF/ID pipeline register are stalled while the ID/EX register is flushed, introducing a bubble into the pipeline.

The unit also assists in branch hazard handling. When a branch or jump instruction is taken, instructions that were fetched sequentially may no longer be valid. In this case, the Hazard Detection Unit generates flush signals to remove these incorrect instructions from the pipeline and allow execution to continue from the correct target address.

Functions of the Hazard Detection Unit

- Detects load-use hazards
- Detects taken branch and jump instructions
- Stalls the Program Counter when required
- Stalls the IF/ID pipeline register
- Flushes incorrect instructions from the pipeline
- Inserts bubbles to ensure correct execution

8. CONTROL SIGNALS

The Control Unit decodes the instruction opcode and generates the control signals required for the execution of different instruction types. These signals control the datapath, memory operations, ALU functionality, and branch execution.

8.1 RegWrite

RegWrite enables writing data into the Register File. It is asserted for instructions that produce a result to be stored in a register.

8.2 MemWrite

MemWrite controls write operations in the Data Memory. It is enabled during store instructions.

8.3 MemRead

Enables reading data from Data Memory for load instructions.

8.4 ALUSrc

ALUSrc selects the second operand of the ALU. The operand can be either a register value or an immediate value depending on the instruction type.

8.5 ResultSrc

ResultSrc determines the source of data to be written back into the Register File. The result may come from the ALU, Data Memory, PC+4, or an immediate value.

8.6 Branch Control Signals

Controls branch and jump instruction execution and Program Counter updates.

8.7 ALUControl

ALUControl specifies the operation to be performed by the ALU, such as addition, subtraction, logical operations, comparison, and shift operations.

8.8 ImmSrc

ImmSrc controls the generation of immediate values. It selects the appropriate immediate format based on the instruction type, such as I-Type, S-Type, B-Type, U-Type, or J-Type.

8.9 WD3Src

WD3Src controls the write-back source for jump instructions. It selects whether the Register File receives the normal execution result or the PC+4 value.

8.10 BranchOp

BranchOp is used for branch and jump instructions. It identifies the branch operation to be performed and helps determine whether the Program Counter should be updated with a target address.

9. COMMON RISC-V INSTRUCTIONS

RISC-V instructions are mainly divided into six types: **R, I, S, B, U, J**

Below are the formats of all 6 types of instructions:

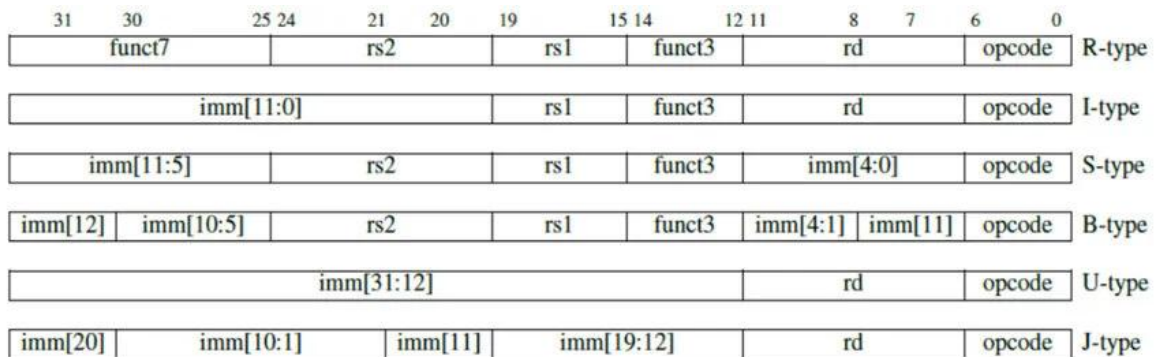


Fig.4: Format of RISC-V Instructions

9.1 R-Type Instructions

Below are the R-Type supported instructions of RISC-V:

ASSEMBLY CODE	INSTRUCTION	DESCRIPTION
ADD rd, rs1, rs2	Add	$rd \leftarrow rs1 + rs2$
SUB rd, rs1, rs2	Subtract	$rd \leftarrow rs1 - rs2$
SLT rd, rs1, rs2	Set less than	$rd \leftarrow rs1 < rs2 ? 1 : 0$
SLTU rd, rs1, rs2	Set less than unsigned	$rd \leftarrow rs1 < rs2 ? 1 : 0$
AND rd, rs1, rs2	AND	$rd \leftarrow rs1 \& rs2$
OR rd, rs1, rs2	OR	$rd \leftarrow rs1 rs2$
XOR rd, rs1, rs2	XOR	$rd \leftarrow rs1 \wedge rs2$
SLL rd, rs1, rs2	Shift left logical	$rd \leftarrow rs1 \ll rs2$
SRL rd, rs1, rs2	Shift right logical	$rd \leftarrow rs1 \gg rs2$
SRA rd, rs1, rs2	Shift right arithmetic	$rd \leftarrow rs1 \gg rs2$

9.2 I-Type Instructions

Below are the I-Type supported instructions of RISC-V:

ASSEMBLY CODE	INSTRUCTION	DESCRIPTION
ADDI rd, rs1, imm12	Add immediate	$rd \leftarrow rs1 + imm12$
SLTI rd, rs1, imm12	Set less than immediate	$rd \leftarrow rs1 < imm12[1:0]$
SLTIU rd, rs1, imm12	Set less than immediate unsigned	$rd \leftarrow rs1 < imm12[1:0]$
ANDI rd, rs1, imm12	AND immediate	$rd \leftarrow rs1 \& imm12$
ORI rd, rs1, imm12	OR immediate	$rd \leftarrow rs1 imm12$
XORI rd, rs1, imm12	XOR immediate	$rd \leftarrow rs1 \wedge imm12$
SLLI rd, rs1, shamt	Shift left logical immediate	$rd \leftarrow rs1 \ll shamt$
SRLI rd, rs1, shamt	Shift right logical immediate	$rd \leftarrow rs1 \gg shamt$
SRAI rd, rs1, shamt	Shift right arithmetic immediate	$rd \leftarrow rs1 \gg shamt$
LW rd, imm12(rs1)	Load word	$rd \leftarrow mem[rs1 + imm12]$
LH rd, imm12(rs1)	Load halfword	$rd \leftarrow mem[rs1 + imm12]$
LB rd, imm12(rs1)	Load byte	$rd \leftarrow mem[rs1 + imm12]$
LWU rd, imm12(rs1)	Load word unsigned	$rd \leftarrow mem[rs1 + imm12]$
LHU rd, imm12(rs1)	Load halfword unsigned	$rd \leftarrow mem[rs1 + imm12]$
LBU rd, imm12(rs1)	Load byte unsigned	$rd \leftarrow mem[rs1 + imm12]$
JALR rd, imm12(rs1)	Jump and link register	$rd \leftarrow PC + 4$ $PC \leftarrow rs1 + imm12$

9.3 S-Type Instructions

Below are the S-Type supported instructions of RISC-V:

ASSEMBLY CODE	INSTRUCTION	DESCRIPTION
SW rs2, imm12(rs1)	Store word	$rs2(31:0) \rightarrow mem[rs1 + imm12]$
SH rs2, imm12(rs1)	Store halfword	$rs2(15:0) \rightarrow mem[rs1 + imm12]$
SB rs2, imm12(rs1)	Store byte	$rs2(7:0) \rightarrow mem[rs1 + imm12]$

9.4 B-Type Instructions

Below are the B-Type supported instructions of RISC-V:

ASSEMBLY CODE	INSTRUCTION	DESCRIPTION
BEQ rs1, rs2, imm12	Branch equal	if $rs1 == rs2$ $PC \leftarrow PC + imm12$
BNE rs1, rs2, imm12	Branch not equal	if $rs1 \neq rs2$ $PC \leftarrow PC + imm12$
BGE rs1, rs2, imm12	Branch greater than or equal	if $rs1 \geq rs2$ $PC \leftarrow PC + imm12$
BGEU rs1, rs2, imm12	Branch greater than or equal unsigned	if $rs1 \geq rs2$ $PC \leftarrow PC + imm12$
BLT rs1, rs2, imm12	Branch less than	if $rs1 < rs2$ $PC \leftarrow PC + imm12$
BLTU rs1, rs2, imm12	Branch less than unsigned	if $rs1 < rs2$ $PC \leftarrow PC + imm12 \ll 1$

9.5 U-Type Instructions

Below are the U-Type supported instructions of RISC-V:

ASSEMBLY CODE	INSTRUCTION	DESCRIPTION
LUI rd, imm20	Load upper immediate	$rd \leftarrow imm20 \ll 12$
AUIPC rd, imm20	Add upper immediate to PC	$rd \leftarrow PC + imm20 \ll 12$

9.6 J-Type Instructions

Below are the J-Type supported instructions of RISC-V:

ASSEMBLY CODE	INSTRUCTION	DESCRIPTION
JAL rd, imm20	Jump and link	$rd \leftarrow PC + 4$ $PC \leftarrow PC + imm20$

10. INSTRUCTIONS SUPPORTED BY OUR DESIGN

Our design supports a total of 26 instructions from the RV32I instruction set, which are:

INSTRUCTION TYPE	INSTRUCTIONS IMPLEMENTED	COUNT
R-Type	ADD, SUB, AND, OR, XOR, SLT, SLL, SRL, SRA	9
I-Type	ADDI, ANDI, ORI, XORI, SLTI, SLLI, SRLI, SRAI, LW, JALR	10
S-Type	SW	1
B-Type	BEQ, BNE, BLT, BGE	4
U-Type	LUI	1
J-Type	JAL	1
Total	26 Instructions	26

11. SIMULATION RESULTS

The following program for finding the maximum value in an array was implemented and tested on the processor along with individual instruction test cases and corner-case verification.

ASSEMBLY CODE	MACHINE CODE
ADDI x1,x0,15	00F00093
SW x1,0(x0)	00102023
ADDI x1,x0,7	00700093
SW x1,4(x0)	00102223
ADDI x1,x0,42	02A00093
SW x1,8(x0)	00102423
ADDI x1,x0,23	01700093
SW x1,12(x0)	00102623
ADDI x1,x0,9	00900093
SW x1,16(x0)	00102823
ADDI x1,x0,55	03700093
SW x1,20(x0)	00102A23
ADDI x1,x0,31	01F00093
SW x1,24(x0)	00102C23
ADDI x1,x0,18	01200093
SW x1,28(x0)	00102E23
ADDI x1,x0,60	03C00093
SW x1,32(x0)	02102023
ADDI x1,x0,27	01B00093
SW x1,36(x0)	02102223
ADDI x1,x0,0	00000093
ADDI x2,x0,10	00A00113
ADDI x3,x0,0	00000193
LW x4,0(x3)	0001A203
LOOP: BEQ x1,x2,DONE	02208463
SLLI x5,x1,2	00209293
ADD x6,x3,x5	00518333
LW x7,0(x6)	00032383
BLT x4,x7,UPDATE	00724663
ADDI x1,x1,1	00108093
JAL x0,LOOP	FE9FF06F
UPDATE: ADD x4,x7,x0	00038233
ADDI x1,x1,1	00108093
JAL x0,LOOP	FDDFF06F
DONE: SW x4,40(x3)	0241A423

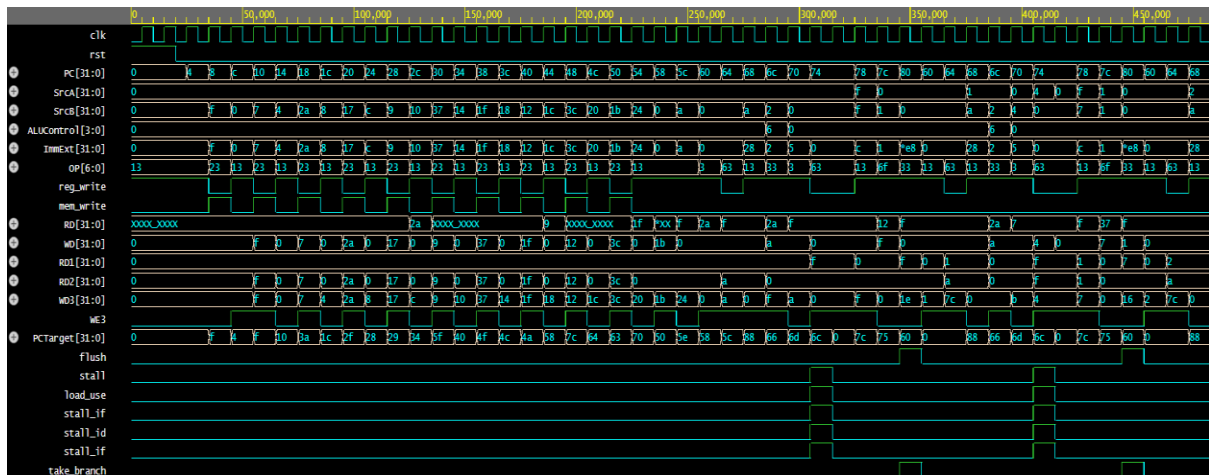


Fig.5: Simulation Result

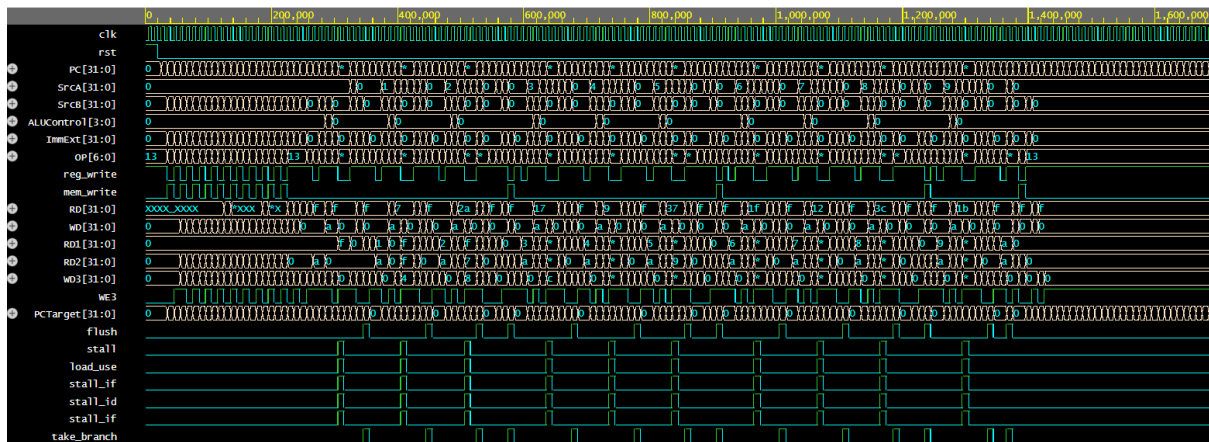


Fig.6: Simulation Result

```

===== FINAL STATE =====
PC = 00000188
x0 = 0
x1 = 10
x2 = 10
x3 = 0
x4 = 60
x5 = 36
x6 = 36
x7 = 27
x8 = 0
x9 = 0
x10 = 0
mem0 = 15
mem1 = 7
mem2 = 42
mem3 = 23
mem4 = 9
mem5 = 55
mem6 = 31
mem7 = 18
mem8 = 60
mem9 = 27
mem10 = 60

```

Fig.7: Contents of registers and memory

The maximum value (60) was successfully identified and stored at memory address 40 (memory location index 10).

12. ADVANTAGES

The proposed pipelined RISC-V processor offers several advantages:

- Increased instruction throughput due to pipelined execution.
- Supports multiple instruction formats of the RV32I instruction set.
- Implements forwarding and hazard detection mechanisms for efficient execution.
- Modular design makes the processor easy to understand and modify.
- Open-source RISC-V architecture eliminates licensing restrictions.
- Better utilization of hardware resources compared to a non-pipelined processor.

13. LIMITATIONS

Despite its advantages, the processor has certain limitations:

- Supports only a subset of the RV32I instruction set.
- Branch prediction is not implemented.
- Cache memory is not included in the design.
- Exceptions and interrupt handling are not supported.
- Branch instructions may introduce pipeline penalties.

14. APPLICATIONS

The proposed processor can be used in various applications, including:

- FPGA-based processor implementation projects.
- Embedded system development.
- IoT and low-power computing applications.
- Digital system design and verification experiments.
- Custom processor development and prototyping.

15. CHALLENGES

The major challenges encountered during the project were:

- Designing the pipelined data path and connecting all stages correctly.
- Generating appropriate control signals for different instruction types.
- Implementing forwarding logic for data hazard resolution.
- Detecting and handling load-use hazards.
- Managing branch and jump instructions in the pipeline.

16. LEARNINGS

During the development of this project, the following concepts were learned and applied:

- RISC-V instruction set architecture (RV32I).
- Five-stage pipelined processor design.
- Datapath and control path implementation.
- Hazard detection and forwarding techniques.
- Pipeline register design.

17. CONCLUSION

This project successfully implemented a 32-bit five-stage pipelined RISC-V processor using Verilog HDL. The processor supports arithmetic, logical, memory access, branch, jump, and immediate-based instructions from the RV32I instruction set. The design incorporates pipeline registers, a Forwarding Unit, and a Hazard Detection Unit to ensure efficient and correct instruction execution.

Simulation results verified the functionality of the processor and demonstrated successful execution of the supported instructions. The project provided practical experience in processor architecture, pipelined design techniques, hazard handling mechanisms, and hardware verification.

18. REFERENCES

- [1] Computer Organization and Design RISC-V Edition, D. A. Patterson and J. L. Hennessy.
- [2] RISC-V International, "RISC-V Specifications".
- [3] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Pearson Education.
- [4] M. Morris Mano and Michael D. Ciletti, *Digital Design*, Pearson Education.

